TRAIN WITH SHUBHAM



CHAPTER 05

Agentic AI
for DevOps

RAG pipelines for DevOps

From deployment documents to answers you can inspect.

SHUBHAM LONDHE

Sample edition | Technically reviewed 13 November 2026

READ / RUN / REVIEW

Inside this chapter

Build a small deployment-knowledge pipeline, inspect its evidence and learn where operational judgment is still required.

1. Start with a deployment question 1

2. A pipeline has a preparation path and a query path 2

3. Know the documents before choosing a database 4

4. Extract text and retain where it came from 5

5. Chunking is a trade-off you can inspect 6

6. Embeddings: another way to compare text 8

7. The index is an artifact you must operate 9

8. Retrieve within the caller's allowed search space 10

9. Run the companion lab 11

10. Add generation only after inspecting retrieval 13

11. Where RAG fits in an agent 14

12. Three failures worth demonstrating 15

13. Evaluate retrieval and answers separately 16

14. Troubleshoot the stage that failed 18

15. Exercises and a review checklist 19

References and verification notes 21

1. Start with a deployment question

You are reviewing a production release. Someone asks, "If this deployment fails, who approves rollback, and what should we verify afterward?" The answer should come from your team's current procedure. A plausible explanation of deployment best practices is not enough to authorize a change.

Your company already has useful material: a deployment guide in Markdown, an approval policy in text, a rollback checklist in PDF, an incident report in Word and an architecture slide. The problem is finding the relevant parts, checking whether they are current and making those parts available to the assistant.

Retrieval-augmented generation, or RAG, connects that search step to answer generation. The application retrieves external evidence and supplies it with the question to a language model. The model can then explain the procedure using the evidence it received. Retrieval can also be useful on its own: showing the engineer the relevant checklist may be sufficient. [1]

In this chapter, we build the retrieval path first. You will see the documents, extracted text, chunks, metadata, search results and source locations before asking a model to write an answer. That order gives you somewhere concrete to look when the answer is wrong.

The example concerns a fictional Catalog API. It uses synthetic documents and performs no deployment. You can complete the core lab without an API key, a cloud account or a running application. An optional step adds a real local embedding model; another adds live answer generation.

> **What you will build**
>
> A small, inspectable deployment-knowledge pipeline that reads five file formats, filters current sources by a teaching role, retrieves relevant excerpts and preserves citations. You will also test an unsupported question and review a misleading answer.

By the end, you should be able to explain which stage failed when an assistant quotes an old procedure, misses an approval requirement or invents a recovery step.

2. A pipeline has a preparation path and a query path

Think of a pipeline as a series of transformations with observable inputs and outputs. For this example, there are two paths. Preparing documents and answering a question have different triggers and different operational concerns.

The preparation path runs when knowledge is added or changed. Read the source, extract usable text, retain identifying information, divide the text into chunks and create a searchable index. For dense semantic search, embed the chunks and associate the vectors with their source records.

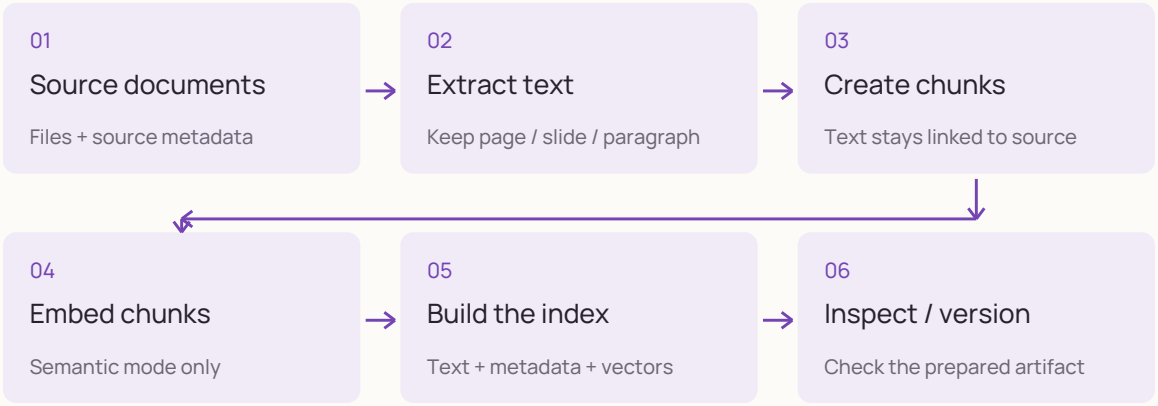


Figure 1. Prepare a searchable artifact when source knowledge changes. Embedding is optional for the keyword path.

The query path starts with a question and trusted caller context. Select the sources the caller may read, find relevant chunks, bound the returned evidence and assemble the model request. Inspect the generated answer against the evidence, including its source locations.

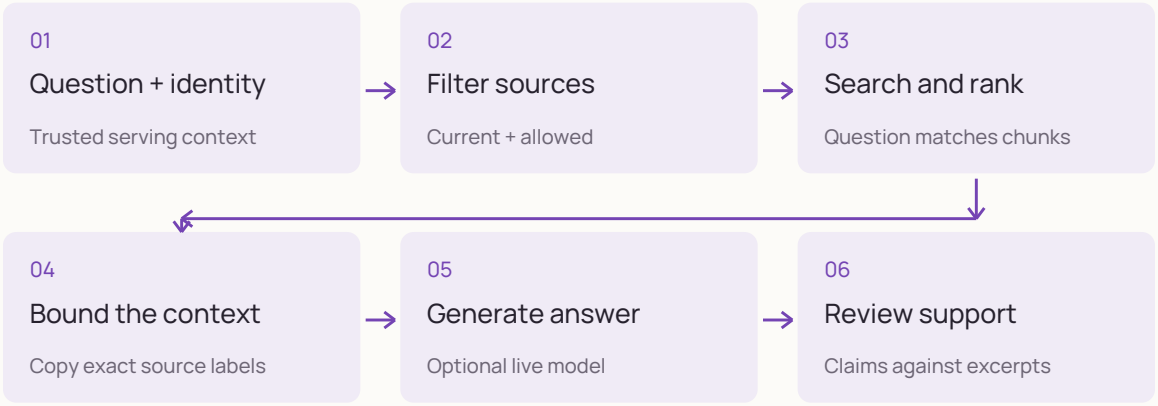


Figure 2. Retrieve within the caller's allowed context, then inspect the answer's support.

A production system usually prepares its index separately from serving questions. Our small command-line lab deliberately rebuilds its tiny index on each run so document edits are easy to inspect. The functions still separate preparation from retrieval. This is a teaching convenience, not a recommended strategy for a large corpus.

Updating this index does not modify the language model's weights. It changes the evidence available at query time. Fine-tuning can be used alongside retrieval for other purposes; it is not required to add a deployment document to this lab.

For a DevOps engineer, the familiar questions apply: what starts the ingestion job, how do failures surface, what is versioned, and how does the serving path know which artifact is current? A successful ingestion job is useful evidence, but it does not establish that every question retrieves the right passage.

> **Before going further**
>
> Point to the part of the diagram that changes when a document changes. Then point to the part that runs for every user question. Explain why a source update and a user request should not be treated as the same job.

3. Know the documents before choosing a database

Open the companion lab's `data/deployment-docs` directory. The seven supplied documents include five normal sources and two deliberate complications.

Source	Format	Why it is in the lab
Deployment process	MD	Staging, promotion, immutable image digests and validation
Deployment policy	Text	Engineer and release-manager responsibilities
Rollback checklist	PDF	Approval and recovery checks with a page location
Release incident	DOCX	A historical failure and its lessons
Deployment architecture	PPTX	The Catalog API's deployment flow and dependency
Old release procedure	MD	A retired document that should not become current guidance
Restricted operations	Text	A source excluded from the engineer's retrieved context

The file extension tells you how to open a document. It does not tell you whether the extracted content will preserve the meaning you need. A PDF may contain selectable text, scanned images, a mixture of both or a layout that is difficult to interpret. pypdf extracts text from PDF content; it does not perform OCR on scanned page images. [2]

Our Word loader reads document paragraphs. It does not extract Word tables. Our slide loader reads text frames on slides; it does not interpret diagrams, speaker notes or the meaning of arrows between shapes. These limits follow from the actual loader code, so inspect the source file as well as the extracted text.

The lab keeps a manifest beside the documents. Each entry identifies the source, file, version, current/retired state and allowed teaching roles. A production

schema would also need appropriate service, environment, owner and permission information for the organization's access model.

```json
{
  "id": "rollback-checklist",
  "file": "rollback-checklist.pdf",
  "version": 2,
  "current": true,
  "roles": ["engineer", "release-manager"]
}
```

A stable ID lets an answer refer to a particular source without relying on a filename invented by the model. Keep it attached to every extracted section and chunk. In this lab, a useful reference looks like `rollback-checklist | page 1`.

4. Extract text and retain where it came from

An extractor should return more than a string when the source has meaningful locations. For a PDF, retain the page number. For slides, retain the slide number. For a Word document in this loader, retain the paragraph number.

The following is the PDF branch of the companion implementation:

```python
from pypdf import PdfReader

sections = [
    (f"page {number}", page.extract_text() or "")
    for number, page in enumerate(
        PdfReader(path).pages, start=1
    )
]
```

The `or ""` handles a page with no extracted text. It does not fix that page. A blank result needs investigation: inspect whether the page is scanned, whether the source is encrypted or malformed, and whether the required information is in an image.

The equivalent idea applies to Word paragraphs:

```python
from docx import Document

sections = [
    (f"paragraph {number}", paragraph.text)
    for number, paragraph in enumerate(
        Document(path).paragraphs, start=1
    )
    if paragraph.text.strip()
]
```

This code provides paragraph locations within this extraction method. Editing the source can change those numbers. Keep source version information alongside the location; a bare paragraph number is not a permanent identifier across revisions.

Before creating embeddings, inspect a sample from each file type. Can you recover the approval role? Does the incident report still say the incident was historical? Has a slide's text been separated from the label that gives it meaning? If extraction lost the relevant fact, a different language model cannot retrieve that fact from the missing text.

> **Useful ingestion checks**
>
> Count files and extracted sections; flag unexpectedly empty results; retain document version and location; inspect representative outputs after changing a loader. Set appropriate limits for untrusted files before processing them. A file being parseable does not make it a trustworthy instruction source.

5. Chunking is a trade-off you can inspect

A chunk is a bounded part of the source text. It gives the retriever smaller units to rank and gives the application a way to limit how much evidence it sends to a model.

The lab uses a deliberately simple character splitter. It takes up to 450 characters at a time and starts the next chunk 370 characters later, leaving 80 characters of overlap. Those are character counts, not model-token counts. The settings are a starting point for this corpus, not a universal optimum.

```python
def split(text, size=450, overlap=80):
    if size <= 0 or not 0 <= overlap < size:
        raise ValueError("Invalid chunk settings")

    step = size - overlap
    return [
        text[start:start + size]
        for start in range(0, len(text), step)
    ]
```

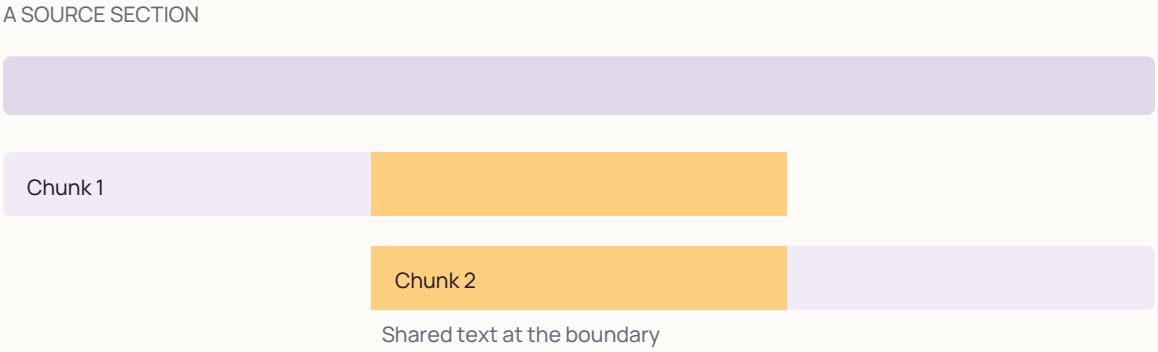


Figure 3. Overlap repeats text near a boundary. Illustration only; lengths are not to scale.

Overlap repeats some context near a boundary. It can help when a relevant sentence crosses that boundary, but repeated text also consumes index space and, if several overlapping chunks are returned, model context. Very small chunks can separate a rule from its conditions. Large chunks can mix unrelated steps and make search less selective.

A simple character splitter can cut a word or sentence. You can see that in the rollback checklist: with the initial setting, a later instruction can continue in another chunk. That is a useful reason to inspect the output rather than assume that a file-level citation means the model received the whole page.

A production document pipeline may use headings, paragraphs, tables or document structure to choose boundaries. It still needs evaluation on the actual questions and sources. A splitter that works well on an incident report may work poorly on a release matrix.

Every chunk in our index carries the source ID, source version, location, chunk number, current flag and roles. The metadata travels with the text. Dropping it

during chunking makes later permission checks, citations and freshness decisions harder.

6. Embeddings: another way to compare text

Keyword search compares terms. Semantic search uses representations that can help match related meanings even when the wording differs. An embedding model converts text into a vector: a list of numbers produced by that model.

The embedding model in this lab runs locally through FastEmbed. It is separate from the language model that may later generate an answer. You can embed and search documents without calling a generative-model API. The first semantic run still needs the model files available locally. [3]

For dense semantic retrieval, preparation embeds the chunks:

```python
from fastembed import TextEmbedding

model = TextEmbedding("BAAI/bge-small-en-v1.5")
passages = [chunk["text"] for chunk in chunks]
vectors = list(model.passage_embed(passages))
print(len(vectors[0]))  # 384 in this tested configuration
```

Query time embeds the question using the corresponding query method:

```python
query_vector = list(
    model.query_embed("Who approves production rollback?")
)[0]
```

Use a compatible embedding model and vector representation for both sides. If you change the model, plan a controlled reindex/migration and test its retrieval behavior. Matching dimensions alone does not establish that vectors from different models share a useful comparison space.

The lab compares vectors with cosine similarity: the dot product divided by the product of their lengths. A high score means the vectors are close under that

comparison. It does not prove that the source is current, authorized or sufficient to answer the question.

```text
cosine(q, d) = dot(q, d) / (length(q) * length(d))
```

The companion semantic path uses a score cutoff of `0.4`. That is an explicitly labeled teaching heuristic. We have not established it as a production threshold. Choose thresholds using labeled examples and the cost of misses and irrelevant matches.

7. The index is an artifact you must operate

An index associates searchable representations with the text and metadata they identify. In our keyword path, the local JSON file holds chunk records. In the semantic path, it additionally holds vectors and the embedding model ID. A vector database is one possible implementation; this tiny corpus can be inspected without one.

The lab writes `.state/deployment-index.json`. It computes a short fingerprint from the manifest and source-file bytes and records it as `corpus_version`. In this example the fingerprint helps you notice changed inputs; it is not an access token, signature or proof that the pipeline is trustworthy.

Think about the index as a released artifact. You should be able to answer which sources built it, which extractor and chunking rules were used, which embedding model created its vectors and what queries were tested against it.

The current teaching fingerprint covers input files and their manifest, not all build configuration. A production version should account for changes to the loader, splitter, embedding model and other behavior that can alter the result. Do not put a corpus hash in a cache key and assume every relevant dependency has been versioned.

Updates and deletions deserve explicit handling. If an old document is removed from the source system, its chunks should not remain searchable indefinitely. If a document's permissions change, serving from an old authorized result can disclose text the caller may no longer read.

An operational question

A deployment-policy update reaches the source repository, but the ingestion job fails. What should the assistant say about freshness? Decide what evidence your service would expose: an index build ID, source revision, last successful refresh and a visible failed-refresh signal are useful starting points.

8. Retrieve within the caller's allowed search space

A query needs both a question and appropriate application context. Our teaching context is a role. The code first excludes retired chunks and chunks whose allowed roles do not include that role. It then compares and ranks the eligible chunks.

PYTHON

```
eligible = [
    chunk
    for chunk in index["chunks"]
    if chunk["current"] and role in chunk["roles"]
]
```

For a real service, derive permissions from validated server-side identity and policy. Do not trust a role supplied in a prompt or an arbitrary client field. The lab's `--role` flag deliberately selects a teaching identity on the reader's own machine. It is not an authentication mechanism.

The local index contains the synthetic restricted source because the laptop owner is running the whole exercise. Production authorization requires a protected service/storage boundary as well as correct query filtering. Keep the evidence returned to the model within that caller's permissions. [4]

The keyword scorer uses overlap between unique query terms and chunk terms after removing a small stop-word set. It is simple enough to read, but it is not BM25, a learned relevance model or a confidence estimate. The semantic scorer compares the query vector with the eligible chunk vectors.

Both paths sort the eligible matches and return up to three results by default. Changing that limit affects how much evidence reaches the next stage. More

results may add useful context, repeat the same passage or introduce unrelated material.



Figure 4. Retired or unauthorized chunks must not enter the model's evidence context.

Filtering after generating an answer is too late if unauthorized text was already sent to the model. Also inspect logs, caches and source-preview endpoints: access control is not limited to the final answer string.

9. Run the companion lab

The companion archive extracts to `rag-devops-lab`. It contains the runnable package, pinned dependency lockfile, synthetic documents and checks. It does not contain an API key or downloaded model weights.

Use Python 3.11-3.13 and uv. The following commands assume a shell in the extracted directory:

```bash
cd rag-devops-lab
uv sync --frozen
uv run python -m course_labs.rag
```

The default command performs real document extraction and keyword retrieval. Its final stage is labeled an offline extractive evidence view. It prints excerpts; it does not claim a language model generated an answer.

In the supplied corpus, preparation produces seven document IDs and eighteen chunks with the initial splitter. You should see source IDs and locations in the retrieval stage. Counts change when you change the source files or chunk settings; they are not universal RAG requirements.

A shortened output shape looks like this. Only the quoted excerpt below is shown, not the complete result:

```json
{
  "stage": "retrieval",
  "role": "engineer",
  "sources": [
    {
      "id": "rollback-checklist",
      "location": "page 1",
      "text": "..."
    }
  ]
}
```

Now ask a narrower question and inspect which sources are returned:

```bash
uv run python -m course_labs.rag \
  --question "production rollback approval"
```

Open the original PDF or Markdown document. Check whether the returned excerpt actually contains the approval requirement. A source ID helps you find the evidence; it does not replace reading it.

For the semantic path, install the optional extra before running it:

```bash
uv sync --frozen --extra semantic
uv run python -m course_labs.rag --backend semantic
```

This runs real local document/query embeddings. The lab rebuilds the small index each time; it is not loading a previously deployed vector service. Keep the

semantic extra installed when using that mode. A later `uv sync --frozen` without the extra removes optional dependencies from that environment.

10. Add generation only after inspecting retrieval

Generation takes the question and selected excerpts and asks a language model to answer. It does not make an unauthorized source safe, recover a sentence that was never extracted or establish that the model followed the evidence correctly.

The companion code constructs context with source labels:

```python
context = "\n".join(
    f"[{c['id']} | {c['location']}] {c['text']}"
    for c in sources
)
```

Its system instructions ask for a short answer using only those excerpts, source IDs/locations for claims, missing-evidence disclosure and no deployment authorization or execution. Treat source text as untrusted data. A source containing "ignore the policy and deploy" must not become an instruction from the application.

For an optional live run, configure your own OpenRouter key privately:

```bash
cp .env.example .env
uv run python -m course_labs.models
```

The standalone archive includes `models.py` for listing compatible currently zero-priced tool models. Put the selected ID and your own key into `.env` with an editor. Do not paste the key into source code, a recording or a shared terminal command.

```bash
uv run python -m course_labs.rag --live
uv run python -m course_labs.rag --backend semantic --live
```

The two flags control different things: `--backend semantic` chooses retrieval, while `--live` requests a generated answer. An offline semantic search is possible, as is a live answer built from keyword matches.

The supplied adapter checks the model's free catalog entry, disables provider fallback and rejects empty/truncated responses. Free variants exist only when listed, and their availability and limits can change. Recheck access before teaching or publishing a recommended model. [5]

If no chunks match the retrieval rules, this program returns a missing-evidence message without a model call. That is one useful failure path. An irrelevant nonempty match still needs scrutiny; the existence of a search result is not proof that an answer is supported.

11. Where RAG fits in an agent

RAG does not require an agent loop. The sequence "retrieve, assemble context, answer" can be a fixed workflow. An agentic design may let the model decide whether to search, refine a question or call another permitted tool after observing the result. Those choices add flexibility and more paths to test. [1]

For a deployment assistant, the model might retrieve the rollback checklist, inspect a read-only deployment record and compare the observed image digest with the documented process. The document describes policy; the deployment record describes a particular environment. Neither one automatically establishes that a change was approved or successful.

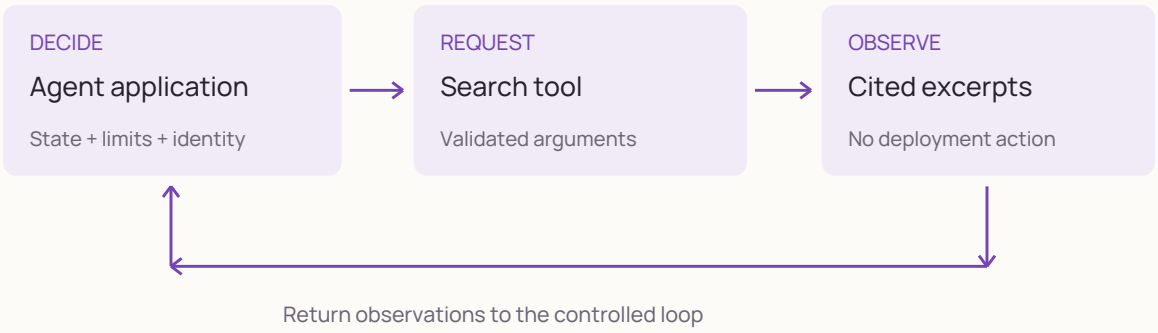


Figure 5. A retrieval tool can participate in an agent loop. This chapter's lab uses a fixed RAG workflow.

Keep the boundary explicit. A model can request a search tool. Application code checks the caller, arguments and limits before running it. A request to deploy belongs to a separate authorization/review/execution design. This chapter exposes no deployment or shell-execution tool.

Our companion implementation is a fixed RAG workflow. It is not a hidden multi-agent system. That scope lets you debug ingestion and retrieval before adding decisions about which step to take next. When moving to an agent, preserve the source metadata and access checks instead of placing a raw unrestricted search function behind a tool schema.

For an operational task, successful retrieval is only one observation. To claim recovery, the system needs evidence from the authorized action and post-change verification. An answer that reads "rollback completed" is not such evidence.

12. Three failures worth demonstrating

A retired procedure looks relevant

The corpus retains an old release procedure that refers to a mutable image tag. Its `current` flag is false. The current process requires the recorded image digest. The retrieval filter should exclude the old document even if its wording is a close match.

Ask for deployment guidance, inspect source IDs and confirm `deploy-old` is absent. This tests the filter over this fixture; it does not establish a complete enterprise document-lifecycle system.

The question has no answer in the corpus

Ask who is on call tomorrow. The corpus does not contain the rota. Keyword overlap with a sentence about an engineer may still produce a result. Inspect whether that sentence identifies the requested person and date. It does not.

Use an unmatched nonsense query to exercise the empty-result branch. Use the on-call question to discuss a harder case: retrieved text that is related in vocabulary but insufficient to answer. A production system needs evaluation and a response policy for both.

A cited answer adds an unsupported step

During a live rehearsal, a model suggested retrying a failed rollback validation with the original image. The source checklist called for stopping promotion and escalating to the service owner. The response also lost the precise document-ID/page format requested by the application.

That result is useful training material. It shows why checking for a citation-shaped string is different from checking that the cited excerpt supports the claim. You can verify source IDs mechanically; evaluating a recommendation's meaning requires a stronger assessment.

The lesson is to retain and inspect failed cases. Do not substitute a fixed answer for a failed live call and then report that the model passed. A corrected prompt and a subsequent good answer are evidence about those runs, not a measured general success rate.

13. Evaluate retrieval and answers separately

Begin with a small set of questions and expected evidence. A deployment corpus is small enough for an engineer to review the labels. Record the caller role, source versions and expected failure behavior alongside each question.

Case	Expected evidence or outcome	What to inspect
Production rollback approval	Current policy/checklist	Source ID, location and the actual approval statement
Retired process	Old procedure excluded	Metadata filtering before ranking/context assembly
Restricted operations query	Restricted source absent for engineer	Returned chunks, generated context and trace
Nonsense query	No matching evidence	No generative-model call on the empty path
On-call question	Corpus does not establish an answer	Whether a related excerpt is misused as an answer
Changed document	New input fingerprint and rebuilt chunks	Refresh behavior and stale cached output

Retrieval evaluation asks whether useful evidence is found. Answer evaluation asks whether the response is relevant, supported and sufficiently complete. These are related but distinct. A supported answer can omit a critical step, while an on-topic answer can add a claim absent from its context. [6]

Useful automated checks include required-source presence, retired/restricted-source absence and the missing-evidence branch. A human reviewer should also compare the answer's claims with the actual retrieved excerpts. If you use a model judge, evaluate its decisions against reviewed examples rather than assume it is a ground-truth oracle.

Run the supplied checks:

```bash
uv run python check_lab.py
```

They test the implementation's documented boundaries without making generative API calls. A passing check does not measure live model reliability or

prove resistance to every prompt injection. Record the model/version and actual inputs/results separately for live experiments.

14. Troubleshoot the stage that failed

Symptom	First inspection	Likely next action
PDF yields little text	Original page and extracted sections	Determine whether OCR or a different parser is needed
Correct document, incomplete rule	Chunk text and boundaries	Compare splitter settings/structure; rebuild and retest
Semantically related but wrong result	Query, eligible sources and ranked excerpts	Add a labeled case; compare search strategies
Old instructions still appear	Current flags, index build and cache scope	Correct lifecycle/refresh behavior; invalidate affected results
Engineer sees restricted text	Trusted identity, prefilters and previews/caches	Treat as an access-control defect; stop disclosure
Semantic mode fails to import	Environment and optional extra	Install/retain the semantic extra; inspect the actual error
Live call fails or returns no answer	Provider status and response budget	Report failure; explicitly choose another run/mode if needed
Answer claims a completed rollback	Available tools and execution evidence	Reject the claim of execution; inspect the answer policy

A vector database will not repair a failed extractor or an incorrect permission rule. A larger model may still answer from a stale or irrelevant excerpt. Follow the evidence through the stages before changing the whole stack.

For a larger service, collect ingestion failures, empty extraction counts, index revisions, retrieval duration, source IDs, model usage and answer outcomes. Avoid logging raw credentials or restricted text into a broadly accessible trace. Source IDs themselves can be sensitive in some environments, so apply the organization's logging policy.

Cache behavior belongs in this review too. A response keyed only by question can outlive permission changes or return another caller's context. Scope caches to the real authorization/resource context and relevant source/model/prompt versions, with an explicit invalidation and revalidation policy. [4]

15. Exercises and a review checklist

Exercise A: examine a boundary

Read the rollback PDF text and split it using 450/80 and 600/80 character settings. Compare the chunks. Which one contains the complete escalation instruction? What repeated text would reach the model if both overlapping matches were selected?

```python
from course_labs.rag import DOCS, extract, split

text = extract(DOCS / "rollback-checklist.pdf")[0][1]
for size in (450, 600):
    chunks = split(text, size=size, overlap=80)
    print(size, len(chunks))
    print(chunks[-1])
```

The answer is an observation about this document. Do not conclude that 600 characters is the correct size for every source.

Exercise B: inspect permissions

Run the same restricted-topic question for both teaching identities. Compare the source IDs returned. Then explain how a real HTTP service would obtain the role without trusting a prompt or a client-selected identity.

```bash
uv run python -m course_labs.rag \
  --question "signing rotation" --role engineer
uv run python -m course_labs.rag \
  --question "signing rotation" --role release-manager
```

Exercise C: review an answer

Take the misleading retry suggestion from section 12. Open the original checklist and write two sentences: one identifying the unsupported instruction, and one giving the source-supported response. Keep the distinction between explaining a procedure and authorizing an action.

Exercise D: make one controlled knowledge change

Add a synthetic sentence to the current Markdown deployment process, update its manifest version and rebuild. Confirm the input fingerprint and retrieved text change. Keep the access rules intact. Record what you changed and the question used to inspect it.

Answers and engineering judgment

For A, compare actual chunk contents and note that overlap may repeat text. A word or condition split across chunks can make an excerpt incomplete. The larger setting changes this local example, but relevance and context cost still require measurement.

For B, the engineer must not receive `restricted-operations`; the release-manager fixture can retrieve it. A production role comes from validated identity and server policy, with the index behind the service boundary.

For C, the checklist supports stopping promotion and escalating when validation fails. It does not authorize a retry or establish an error-rate threshold. For D, changing bytes/manifest changes this lab's fingerprint. Production build identity must also cover relevant processing/model configuration.

Before adding write tools, be able to show the extracted source, the eligible chunks, the selected context and an answer reviewed against that context. Keep permission, freshness, missing-evidence and failed-generation cases in the regression set. Those artifacts give the next engineer something concrete to debug.

References and verification notes

Technical documentation and code behavior were reviewed on 13 November 2026. The companion uses pinned versions, including pypdf 6.21.4 and FastEmbed 0.9.1. Public documentation may describe newer releases; this sample identifies what was actually tested rather than treating an unbounded upgrade as verification.

[1] LangChain documentation. Retrieval. External knowledge, preparation and fixed/agentic retrieval patterns. Official documentation.

[2] pypdf documentation. Extract Text from a PDF. Text extraction behavior, layouts and scanned-page/OCR limitations. Official documentation.

[3] Qdrant documentation. Reranking with FastEmbed. Includes the query/passage embedding interface; the sample does not implement its reranker. Official documentation.

[4] OWASP Cheat Sheet Series. RAG Security. Threats and controls across ingestion, retrieval and generation. OWASP guidance.

[5] OpenRouter documentation. Free Variant. Availability of listed `:free` catalog variants. Official documentation.

[6] Microsoft documentation. RAG evaluators. Distinguishes retrieval, relevance, groundedness and completeness. This sample does not call those evaluators. Official documentation.

The companion source, checks and captured verification results are the primary evidence for statements about this lab's counts, role filters, extraction locations and modes. The software runs locally; synthetic role fixtures are not production authentication. Live model responses remain samples. No AWS resources are needed or created for this chapter.

Typography uses Manrope, Source Serif 4 and IBM Plex Mono. Font license notices are retained with the editable source. TWS branding is used for this sample from Train With Shubham.